

# Phase 5

## Database Architecture

Preferred covered reading edition

**APPROVED SCHEDULING AMENDMENT | PRESERVED REVIEW FOLLOWS**

Read amendment first; underlying phase approval status is retained

# Aesthenda Core Phase 5 Scheduling Amendment

**Status:** APPROVED DECISION PROPAGATION

**Effective:** September 7, 2026

This amendment applies DG-02, DG-03, and the approved overlap presentation to Phase 5. It controls conflicting scheduling language in the preserved review package; it does not grant new approval to unrelated material.

## Fixed booking holds

All businesses use a fixed ten-minute booking hold, timed by the server from hold creation. There is no business, Service, or Appointment duration setting. Release early on explicit cancel/close; otherwise expire automatically. Confirmation revalidates the hold and current scheduling state; an expired hold cannot confirm from stale client state.

## Professional overlap exceptions

Public booking never offers or confirms a non-fitting hands-on overlap. The authenticated owner-professional may manually book it after a specific warning and explicit confirmation bound to the exact request and current affected revisions. Published policy alone cannot authorize a hands-on conflict. Record the warning, actor, time, affected Appointments and segments, policy version, confirmation, and outcome. Preserve the full booked duration unless the professional explicitly edits it; never silently shorten work or move an existing appointment.

## Minimal calendar presentation

Shade only the exact conflicting interval amber on both affected appointments, keeping their normal colors elsewhere. Show the same highlight while placing or moving the appointment, before the confirmation warning, and retain it after approval while the conflict exists. Use no persistent overlap label or extra icon. Tap or click reveals the overlap duration, affected appointments, and approval details. Provide the same details through keyboard interaction and screen-reader text without adding visible clutter.

## Approved example

With zero buffers and other rules satisfied, a 10:00-10:15 application, 10:15-10:45 released processing segment, and 10:45-11:30 haircut permit a fitting 30-minute public service in the processing gap. A 45-minute service at 10:15 ends at 11:00 and is not offered publicly. A manual booking previews amber from 10:45 to 11:00 on both appointments, warns of the 15-minute overlap, and requires Book anyway before saving. Go back returns to editing without committing. Approval preserves the 45-minute booking and existing haircut timing.

## Database effects

P5-D05 remains a fixed ten-minute server hold for every business. Persist server creation and expiry facts, status, release time/reason, and session/account ownership. No business policy or service field controls hold duration. A repeated idempotent creation does not extend the existing hold.

Persist appointment and segment timing as booked, including the 45-minute service in the approved example. Extend the overlap-decision record with exact conflict intervals, affected appointment/segment revisions, warning basis, actor, confirmation time, policy version, and outcome. These facts support details on demand; color is presentation rather than database truth.

Supersede the Section 6 claim that published policy may itself permit hands-on overlap. Require a separate case-specific professional confirmation. Section 12 exclusion constraints must preserve genuinely unconditional constraints and must not impose a blanket same-professional hands-on exclusion that rejects approved exceptions. Transactional Scheduling checks, scoped locks, and revalidation must reject unconfirmed/public conflicts while allowing properly evidenced professional exceptions. Never disable constraints globally to book an exception. Same-client simultaneous appointments and account isolation remain hard boundaries.

## Verification requirements

Verify ten-minute holds across businesses, early release, stale expiry rejection, public omission/rejection of the 45-minute example, rejected unconfirmed manual save, accepted confirmed save, unchanged durations, and rejection of stale confirmation after a concurrent revision. Verify exact interval rendering before and after approval with details available on demand. These are design acceptance requirements; no application implementation is present in this document library.

# Phase 5 Database Architecture

v2.2 REVIEW DRAFT

**Purpose**

Translate the approved product domain and Phase Four system boundaries into an isolated, auditable, evolvable data model that is ready for schema implementation review.

| Governing scope  | This deliverable                                                                                         |
|------------------|----------------------------------------------------------------------------------------------------------|
| Product          | One independent beauty/wellness professional per business account; many clients.                         |
| Excluded from v1 | Teams, staff, front-desk roles, multi-professional scheduling, payroll, commissions, shared staff books. |
| Architecture     | Modular monolith; canonical actions only; no UI, AI, voice, or integration database access.              |
| Status           | APPROVED FOR PHASE SIX - design authority only; not production authorization.                            |

Prepared under Aesthenda Core v2.2 Master Blueprint

September 1, 2026

# 1. Executive Summary

This design makes the business account the tenant boundary, preserves historical booking facts, and gives scheduling enough structure to support intentional overlap without forcing any operational model on the professional.

## Central approval test

Can automated tests prove that one professional's protected business and client data cannot leak into another account?

## Design commitments carried forward from Phase Four

- **Intentional overlap is first-class.** Double-booking is represented by explicit policy and timed service segments - never hidden conflict overrides.
- **v1 remains time-based.** Stations, rooms, waiting areas, equipment, and named/pool resource capacity are not tracked in v1. Removing them does not weaken segment-based scheduling.
- **Canonical actions own writes.** Web, mobile, voice, AI, and integrations call the same application actions. Database credentials are never issued to those surfaces.
- **Boundaries remain visible.** Policy, scheduling, appointments/bookings, payments, themes, audit, integrations, and agents own separate tables and write paths inside one v1 database.
- **Payment data stays minimized.** Providers process sensitive payment data. Aesthenda stores references, states, amounts, attempts, refunds, failures, reconciliation, and audit history.
- **Branding is bounded.** Versioned safe theme tokens and assets may alter presentation, never canonical services, prices, timing, policies, consent facts, labels, or booking flow.

## Approval meaning

This document is approved as binding input to Phase Six API & Action Design. It does not authorize production implementation, vendor selection, production data processing, launch, or final physical migrations.

## 2. Scope, Terms, and Design Rules

### v1 cardinality

| Relationship                                       | v1 rule                                                       | Future-safe posture                                                                                                           |
|----------------------------------------------------|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <b>Business account -&gt; professional profile</b> | Exactly one active professional profile per business account. | A unique account key enforces v1. Future team support requires a later approved migration, not unused staff tables now.       |
| <b>Business account -&gt; clients</b>              | Many client relationships.                                    | A person may later relate to multiple businesses, but protected professional notes and booking history remain account-scoped. |
| <b>Business account -&gt; locations</b>            | One primary operating location/configuration in v1.           | Keys allow a location record without exposing multi-location workflows.                                                       |
| <b>Appointment -&gt; professional</b>              | Implicitly the account owner in v1.                           | No staff assignment table or provider roster in Phase Five.                                                                   |

### Data classification

- **Protected account data:** client identity/contact, professional notes, appointments, policies, consents, payment facts, messages, availability, and unpublished theme assets.
- **Public published data:** approved booking-page projection only; still account-owned and versioned.
- **Immutable evidence:** audit events, financial ledger facts, consent evidence, sent-message facts, and snapshots referenced by completed or consequential actions.
- **Operational transient data:** idempotency records, outbox deliveries, webhook receipts, retry state, locks, and short-lived availability caches.

### Non-negotiable rules

- Every protected row carries **business\_account\_id** directly unless it is a private child whose account is provable through a non-null parent foreign key; direct scoping is preferred for enforcement and indexing.
- Internal identifiers use approved opaque sortable UUIDs (UUIDv7 or equivalent). Friendly human references are separate and never authorize access.
- Money is stored as integer minor units plus ISO currency; timestamps are UTC instants plus saved local-zone context where human intent matters.
- No cross-account foreign key relationship is valid. Composite foreign keys include **business\_account\_id** where practical.

### 3. Conceptual Data Model

The conceptual model groups records by the module that owns their lifecycle. Arrows indicate the principal parent relationship; most child rows also carry `business_account_id` for tenant enforcement.

| Module                | Principal relationships                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| IDENTITY / ACCOUNT    | business_account -> professional_profile; business_account -> account_location; business_account -> account_member (owner only in v1)   |
| CLIENT                | business_account -> client; client -> client_contact_point; client -> client_note; client -> client_merge_record                        |
| OFFER / POLICY        | business_account -> service; service -> service_version -> service_segment_definition; business_account -> policy_set -> policy_version |
| TIME / AVAILABILITY   | business_account -> availability_rule_set -> availability_rule / availability_exception; service_version -> timed segment definitions   |
| BOOKING / APPOINTMENT | booking_session -> appointment -> appointment_segment; appointment -> appointment_snapshot / status_event / note / consent_record       |
| PAYMENT               | appointment -> payment_order -> payment_attempt -> payment_event; payment_order -> refund                                               |
| COMMUNICATION         | appointment/client -> communication -> communication_delivery; template_version -> communication                                        |
| THEME                 | business_account -> theme_config -> theme_version -> theme_asset_link; asset -> asset_version                                           |
| INTEGRATION           | business_account -> integration_connection -> webhook_receipt / sync_cursor / external_reference                                        |
| PLATFORM EVIDENCE     | canonical_action -> idempotency_record / audit_event / outbox_event; agent_invocation -> canonical_action                               |

**Historical truth pattern**

Mutable working records point to immutable versions or embed immutable snapshots. An appointment references the `service_version` and `policy_version` used, then stores a compact booking snapshot of name, duration, price, currency, cancellation terms, timezone, and consent facts. Later catalog edits do not rewrite history.

## 4. Module Ownership and Write Boundaries

| Owner                         | Tables                                                                                            | Owns writes to                                 | May publish/read              |
|-------------------------------|---------------------------------------------------------------------------------------------------|------------------------------------------------|-------------------------------|
| <b>Identity &amp; Account</b> | business_account, professional_profile, account_location, account_member                          | Account lifecycle and owner identity           | All modules                   |
| <b>Client</b>                 | client, client_contact_point, client_note, client_merge_record                                    | Client identity, account-private notes, merges | Booking, communication        |
| <b>Business Policy</b>        | policy_set, policy_version, policy_rule                                                           | Policy drafting/versioning/publish             | Scheduling, booking, payment  |
| <b>Scheduling</b>             | availability_*, overlap_policy_version, slot_hold                                                 | Availability, timed overlap, and hold truth    | Booking                       |
| <b>Catalog</b>                | service, service_version, service_segment_definition                                              | Service facts and timed work model             | Scheduling, booking           |
| <b>Appointment / Booking</b>  | booking_session, appointment, appointment_segment, appointment_status_event, appointment_snapshot | Booking transaction and appointment lifecycle  | Payment, communication        |
| <b>Payment</b>                | payment_order, payment_attempt, payment_event, refund, reconciliation_record                      | Provider-linked financial state                | Appointment read-only outcome |
| <b>Theme</b>                  | theme_config, theme_version, asset, asset_version, theme_asset_link, publication                  | Safe presentation configuration                | Booking-page read projection  |
| <b>Integration</b>            | integration_connection, webhook_receipt, external_reference, sync_cursor                          | External boundary state                        | Canonical actions only        |
| <b>Audit / Reliability</b>    | audit_event, outbox_event, outbox_delivery, idempotency_record                                    | Evidence, events, retries                      | Append-only                   |
| <b>Agent Orchestration</b>    | agent_invocation, agent_tool_call, action_actor_context                                           | Agent provenance and scoped action requests    | Canonical actions only        |

### Boundary enforcement

- Application modules use repository interfaces; table writes outside the owning module fail architecture tests and code review policy.
- Database roles are separated for migrations, application runtime, background workers, read-only operations, and break-glass administration.
- Agents and integrations receive scoped application capabilities, never SQL credentials, generic query tools, or unrestricted record identifiers.

## 5. Table and Key Outline - Account, Client, Catalog, Policy

| Table                             | Key / representative columns                                                                | Rule                                                                                   |
|-----------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>business_account</b>           | id PK; status; timezone; currency; created_at                                               | Tenant root. One active owner/professional enforced for v1.                            |
| <b>account_member</b>             | (business_account_id, id) PK; user_id; role; status                                         | v1 role CHECK owner only; UNIQUE active owner per account.                             |
| <b>professional_profile</b>       | business_account_id PK/FK; display fields; locale                                           | Exactly one per account; protected vs public fields projected separately.              |
| <b>account_location</b>           | (business_account_id, id) PK; timezone; address fields                                      | One active primary location in v1 via partial UNIQUE.                                  |
| <b>client</b>                     | (business_account_id, id) PK; display_name; lifecycle_status; merged_into_id                | Account-local client identity; merge target same account; no silent hard delete.       |
| <b>client_contact_point</b>       | (business_account_id, id) PK; client_id; type; normalized_value; verified_at                | Composite FK to client; indexes scoped by account; sensitive values protected.         |
| <b>client_note</b>                | (business_account_id, id) PK; client_id; author_actor_id; body_ciphertext                   | Account-private; audited; encrypted field/volume as approved.                          |
| <b>service</b>                    | (business_account_id, id) PK; lifecycle_status; current_version_id                          | Stable identity for a service offering.                                                |
| <b>service_version</b>            | (business_account_id, id) PK; service_id; version_no; name; price_minor; currency; duration | Immutable after publish; UNIQUE service/version; CHECK non-negative money/time.        |
| <b>service_segment_definition</b> | (business_account_id, id) PK; service_version_id; ordinal; kind; offset; duration; hands_on | Immutable timed segment: active, processing, cleanup, buffer, or custom approved type. |
| <b>policy_set</b>                 | (business_account_id, id) PK; kind; current_version_id                                      | Stable policy identity: booking, cancellation, payment, overlap, communication.        |
| <b>policy_version</b>             | (business_account_id, id) PK; policy_set_id; version_no; state; effective_at; document_hash | Immutable when published; appointment references exact version.                        |
| <b>policy_rule</b>                | (business_account_id, id) PK; policy_version_id; rule_type; typed_value/json                | Validated typed rules; JSON only for bounded extension, with schema_version.           |

## 6. Scheduling, Availability, and Timed Segments

Scheduling evaluates the professional's working-time rules, exceptions, existing appointment segments, and explicit overlap policy. v1 does not track stations, rooms, waiting areas, equipment, named resources, or resource pools. It returns explainable time-fit/conflict results without encoding a preferred business practice.

| Table                         | Key / columns                                                                                  | Purpose / invariant                                                                                              |
|-------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>availability_rule_set</b>  | (account,id); state; timezone; version                                                         | Versioned container for recurring availability.                                                                  |
| <b>availability_rule</b>      | (account,id); rule_set_id; weekday; local_start/end; effective range                           | Recurring local-time windows; end > start; DST resolved by saved timezone policy.                                |
| <b>availability_exception</b> | (account,id); local date/range; kind; reason                                                   | Close, open, or override windows; no cross-account reference.                                                    |
| <b>overlap_policy_version</b> | (account,id); state; scope; max_concurrent_clients; rule_json/schema                           | Published immutable policy; professional configures maximum clients overlapping during eligible processing time. |
| <b>slot_hold</b>              | (account,id); booking_session_id; starts/ends; expires_at; released_at; release_reason; status | Ten-minute hold. Release immediately when client cancels/closes; otherwise expire and reopen the slot.           |
| <b>appointment_segment</b>    | (account,id); appointment_id; kind; starts_at; ends_at; hands_on; source_definition_id         | Materialized scheduled segments; immutable timing version per appointment revision.                              |

### Time-fit and overlap rules

- **Hands-on exclusion:** by default, active hands-on segments for the account owner may not overlap. A published overlap policy may permit an intentional overlap; the decision and policy version are recorded.
- **Public booking:** automatically block an overlap when it would exceed the professional-configured maximum overlapping clients or when the full service duration does not fit inside an eligible processing segment.
- **Professional authority:** when manually adding outside normal fit or concurrency rules, show a clear warning and require explicit confirmation before saving. Record warning basis, actor, confirmation, affected segments, and audit evidence.
- **Database protection:** PostgreSQL range/exclusion constraints protect unconditional no-overlap cases. Conditional segment-fit policy is enforced transactionally by Scheduling with advisory/row locks, serializable retry, and a post-write invariant check.
- **Explainability:** every proposed overlap returns the eligible processing window, exact fit or overflow, warnings, and blocking conflict before confirmation.

## 7. Booking and Appointment Records

| Table                               | Key / representative columns                                                                                          | Rule                                                                                                                  |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>booking_session</b>              | (account,id); actor/session; expires_at; source_channel; status                                                       | Groups selection, consent, checkout, and a ten-minute hold; cancel/close releases early.                              |
| <b>appointment</b>                  | (account,id); client_id; status; starts/ends; timezone; service_version_id; policy_version_ids; revision              | Canonical appointment. Composite FKs enforce same account. Optimistic revision prevents lost updates.                 |
| <b>appointment_snapshot</b>         | (account,appointment_id,revision) PK; facts_json; schema_version; hash                                                | Immutable service/price/timing/policy/consent presentation facts for each consequential revision.                     |
| <b>appointment_status_event</b>     | (account,id); appointment_id; from/to; occurred_at; actor; reason                                                     | Append-only lifecycle evidence; transition table validates allowed state changes.                                     |
| <b>appointment_note</b>             | (account,id); appointment_id; visibility; ciphertext; author                                                          | Private professional note or approved client-visible note; never public by accident.                                  |
| <b>booking_source</b>               | (account,appointment_id) PK; channel; integration/agent refs; campaign fields                                         | Origin and provenance without granting source write authority.                                                        |
| <b>appointment_overlap_decision</b> | (account,id); appointment_id; policy_version_id; warning_basis; actor; confirmed_at; affected_segments_json; decision | Records the warning, explicit professional confirmation, affected segments, and audit evidence for manual exceptions. |

### Appointment transaction

- Resolve authenticated actor and business\_account\_id; authorize capability before accepting record IDs.
- Load published service/policy/theme facts by composite account key; calculate time-segment fit in the same account.
- Acquire scoped scheduling locks; re-evaluate availability and public processing-segment fit at commit time; consume or release the ten-minute hold.
- Confirm only after the professional's required payment terms are satisfied: deposit, full prepayment, no prepayment, or provider-held card on file.
- Insert appointment, materialized segments, snapshot, payment-policy result, status event, audit event, and outbox events in one transaction.
- Return stable IDs and revision. Retried requests return the original result through the idempotency record.

## 8. Payments, Refunds, and Provider Reconciliation

### Sensitive payment boundary

Aesthenda does not store PAN, CVV, bank credentials, or provider secrets in business tables.

Tokenization/processing stays with the payment provider. Provider references are treated as sensitive opaque identifiers.

| Table                         | Key / columns                                                                                           | Invariant                                                                                                            |
|-------------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>payment_policy_version</b> | (account,id); state; mode; deposit amount/percent; card_on_file_required; effective_at                  | Professional-configurable: deposit, full prepay, no prepay, or provider-held card on file; immutable when published. |
| <b>payment_order</b>          | (account,id); appointment_id; amount_minor; currency; purpose; status; idempotency_key                  | Aesthenda intent for deposit/full payment/adjustment; amount >= 0; currency fixed after attempt.                     |
| <b>payment_attempt</b>        | (account,id); order_id; provider_connection_id; provider_intent_ref; status; started/completed          | One provider attempt; UNIQUE(provider connection, provider reference).                                               |
| <b>payment_event</b>          | (account,id); attempt_id; provider_event_ref; type; occurred/received; payload_digest; normalized facts | Append-only normalized transition; raw payload separately protected/short-retained if needed.                        |
| <b>refund</b>                 | (account,id); order_id; provider_ref; amount_minor; status; reason; requested_by                        | Sum of successful refunds <= captured amount; append status events.                                                  |
| <b>reconciliation_record</b>  | (account,id); provider_connection_id; period/ref; expected/observed; state                              | Detects disagreement; never rewrites prior payment events.                                                           |

### Reliability rules

- Booking confirmation occurs only after the selected professional policy is satisfied. No-prepay confirms without a payment order; deposit/full-prepay require provider success; card-on-file requires a valid provider reference, never raw card data.
- Every provider creation/refund call uses a stable Aesthenda idempotency key and records request outcome before retry.
- Webhook receipts are deduplicated by provider connection + provider event reference; invalid signatures are rejected and audited.
- Out-of-order events are retained, normalized, and reconciled using provider event time plus legal state-transition rules.
- Financial facts are corrected with new events/adjustments, never destructive updates.
- Appointment payment state is a projection derived from payment records, not an independently editable field.

## 9. Integrations, Audit, Agents, Consent, and Communications

| Table                         | Key / representative columns                                                                                             | Rule                                                                                                            |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>integration_connection</b> | (account,id); provider; status; encrypted_secret_ref; scopes; external_account_ref                                       | Account-owned connection; secrets in secret manager; least privilege.                                           |
| <b>webhook_receipt</b>        | (account,id); connection_id; external_event_ref; received_at; signature_status; digest; processing_state; raw_expires_at | Deduplicated intake; encrypted raw payment payload retained 90 days, then deleted; normalized evidence remains. |
| <b>external_reference</b>     | (account,id); connection_id; local_type/id; external_type/id                                                             | UNIQUE within connection/type/ref; same-account local target.                                                   |
| <b>audit_event</b>            | id PK; account_id; action; actor_type/id; target; occurred_at; request/correlation; before/after digests; outcome        | Standard append-only v1 history; protected from application updates/deletes; retained 7 years.                  |
| <b>agent_invocation</b>       | (account,id); agent_type/version; initiator; purpose; capability_scope; input/output digests; outcome                    | Proves why an agent acted and under whose authority.                                                            |
| <b>agent_tool_call</b>        | (account,id); invocation_id; canonical_action; request/result digests; status                                            | No raw DB tool; links to action audit and idempotency.                                                          |
| <b>consent_record</b>         | (account,id); client_id; appointment_id; consent_type; text/version/hash; presentation; decision; occurred_at; actor     | Immutable evidence of exact content and affirmative/withdrawn decision.                                         |
| <b>communication</b>          | (account,id); client/appointment; type; channel; template_version_id; content_digest; requested_by; state                | Canonical intent and rendered-content evidence.                                                                 |
| <b>communication_delivery</b> | (account,id); communication_id; provider_ref; attempt_no; status; timestamps; failure_code                               | Append-only attempts; provider callbacks deduplicated.                                                          |

### Privacy and content minimization

- Audit stores necessary facts and digests, not full sensitive payloads by default.
- Communication bodies and client notes use field or volume encryption appropriate to the deployment; search indexes must not leak plaintext.
- AI transcripts are stored only when needed for audit/support, redacted, retained 3 years, and prohibited from secondary AI training.

## 10. Theme Configuration, Assets, and Publication

| Table                           | Key / columns                                                                                   | Rule                                                                                     |
|---------------------------------|-------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <b>theme_config</b>             | business_account_id PK; current_draft_id;<br>published_version_id                               | One theme config per account.                                                            |
| <b>theme_version</b>            | (account,id); version_no; state; token_schema_version;<br>tokens_json; created_by; published_at | Immutable after publish; validated allowlist of colors, fonts, controlled sizes/density. |
| <b>asset</b>                    | (account,id); kind; lifecycle_status; current_version_id                                        | Logo/image identity; private until referenced by published config.                       |
| <b>asset_version</b>            | (account,id); asset_id; storage_key; mime; bytes; dimensions;<br>digest; scan_status            | Immutable binary metadata; malware/type/size checks; scoped object-store key.            |
| <b>theme_asset_link</b>         | (account,theme_version_id,role) PK; asset_version_id;<br>crop/alt text                          | Only approved roles; same-account composite FKs.                                         |
| <b>booking_page_publication</b> | (account,id); theme_version_id; service/policy projection<br>versions; state; published_at      | Atomic published bundle; rollback points to prior immutable publication.                 |

### Protected presentation contract

- Tokens are allowlisted and typed. No arbitrary CSS, scripts, HTML, remote fonts, layout templates, or hidden elements.
- Core flow, labels, service facts, price, duration/timing, policies, payment state, and consent content render from canonical records and cannot be overridden by theme data.
- Contrast, font availability, controlled sizing ranges, asset safety, responsive behavior, and reduced-motion behavior are validated before publication.
- Preview uses an unpublished immutable version. Publish atomically swaps the public pointer and records actor/audit evidence.

# 11. Tenant Isolation and Authorization

The application authorization layer decides whether the authenticated actor may perform a canonical action. Database isolation independently constrains which account rows the runtime transaction can see or change.

| Layer                              | Control                                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Request context</b>          | Authenticate actor; resolve business_account_id server-side; reject caller-supplied scope that disagrees.                                               |
| <b>2. Capability authorization</b> | Canonical action checks actor type, owner role, channel/integration/agent scopes, target ownership, and state.                                          |
| <b>3. Repository API</b>           | Every query accepts an AccountId value type and uses composite account keys. No unscoped find-by-id methods.                                            |
| <b>4. Transaction context</b>      | Set transaction-local app.business_account_id and actor context from trusted middleware; connection pooling must reset context.                         |
| <b>5. Approved RLS baseline</b>    | PostgreSQL policies require business_account_id = trusted transaction setting for SELECT/INSERT/UPDATE/DELETE. FORCE RLS applies to the runtime owner.  |
| <b>6. Composite integrity</b>      | Child FK (business_account_id,parent_id) references parent (business_account_id,id); cross-account linkage is structurally impossible.                  |
| <b>7. Operational separation</b>   | Migration and break-glass roles bypass only under controlled, logged procedures. Backups, logs, analytics, and object storage retain tenant safeguards. |

## Authorization posture

- The single professional owner may administer that business account. Clients may access only explicitly exposed self-service records through narrowly scoped actions and possession/authentication checks.
- Public booking reads use a deliberately published projection, not broad RLS access to internal tables.
- Background workers claim outbox jobs under platform control, then set and verify the event account context before business-row access.
- Support has no default account access. Break-glass requires explicit professional consent, a recorded reason, a short time limit, and a complete append-only audit trail.

## 12. Constraints, Indexes, and Exclusion Rules

| Area                        | Physical rule / index                                                                                                          | Why                                                                                                               |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>Tenant integrity</b>     | Composite UNIQUE(account,id) on parent; composite FK on children; NOT NULL account_id on protected tables.                     | Prevents cross-account relationships even if application code is wrong.                                           |
| <b>Temporal</b>             | end > start; non-negative durations; effective_to > effective_from; non-overlapping published effective ranges where required. | Rejects impossible intervals.                                                                                     |
| <b>Money</b>                | amount_minor >= 0; ISO currency length/domain; refund aggregate protected transactionally.                                     | Rejects invalid financial facts.                                                                                  |
| <b>Versioning</b>           | UNIQUE(account,stable_id,version_no); published rows immutable trigger/permissions; one published/current pointer.             | Preserves historical truth.                                                                                       |
| <b>Scheduling exclusion</b> | GiST on account + tstzrange for unconditional hands-on no-overlap and active holds, with partial predicate.                    | Hard protection where policy is unconditional.                                                                    |
| <b>Segment fit</b>          | Index appointment segments by account, starts/ends, kind, and status; transactional fit check + locks.                         | Public overlap must fit wholly within an eligible processing segment; manual exceptions are explicit and audited. |
| <b>Idempotency</b>          | UNIQUE(account,action_name,idempotency_key); request_hash; result reference; expiry.                                           | Same key+same request returns same result; changed request conflicts.                                             |
| <b>Provider dedupe</b>      | UNIQUE(connection_id, provider_event_ref/provider_object_ref).                                                                 | Makes webhook and payment retries safe.                                                                           |
| <b>Client lookup</b>        | Account-scoped indexes on normalized contact hashes, display name/search projection, lifecycle status.                         | No global client directory or cross-tenant lookup.                                                                |
| <b>Operational</b>          | Indexes on account+status+scheduled time, outbox available_at/status, audit account+occurred_at, publication pointers.         | Fast common paths; avoid unbounded JSON scans.                                                                    |

### Implementation note

A database exclusion constraint alone cannot express public processing-segment fit plus professional-approved exceptions. The safe design combines hard constraints for unconditional invariants with a locked, serializable Scheduling transaction for policy-dependent checks. A public request fails unless the full service fits; only an authenticated professional action may record a warned manual exception.

## 13. Historical Truth and Versioning

| Domain              | Version pattern                                                                                              | Historical guarantee                                                  |
|---------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Services</b>     | Draft mutable; published service_version immutable.                                                          | Appointment references version and stores snapshot.                   |
| <b>Policies</b>     | Draft mutable; published policy_version immutable/effective-dated.                                           | Appointment/consent/payment references exact version.                 |
| <b>Appointments</b> | Current row plus monotonic revision; append status events and immutable snapshots for consequential changes. | Move/cancel does not erase what was booked or communicated.           |
| <b>Payments</b>     | Append attempts/events/refunds/reconciliation; derived current projection.                                   | Corrections add facts; never rewrite settlement history.              |
| <b>Consent</b>      | Exact text/version/hash, presentation channel, decision, actor, time.                                        | Withdrawal is a new event, not deletion of prior evidence.            |
| <b>Theme</b>        | Draft versions; immutable published version and publication bundle.                                          | Rollback repoints to a known version; booking facts remain canonical. |
| <b>Audit</b>        | Append-only normalized evidence with correlation and actor provenance.                                       | Before/after digests or approved safe deltas; no secrets by default.  |

### Snapshot discipline

- Snapshots contain only facts required to explain the transaction later; they are schema-versioned and hashed.
- A snapshot is not an authorization source and never replaces canonical current records for new actions.
- Display strings shown at booking/consent time are preserved where necessary to prove what the client saw.
- Timezone name, local intended time, UTC instant, and offset are retained for booked events to survive DST and future timezone-rule changes.

# 14. Retention, Archive, Deletion, and Anonymization

The following approved v1 periods are measured from the last relevant appointment or record event. A documented legal or dispute hold suspends deletion/anonymization only for the affected records and period.

| Data class                             | Retention basis                                                        | End-of-life action                                                                         | Guardrail                                                                    |
|----------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Account/profile/config</b>          | While active; closure handling follows account workflow                | Revoke connections/assets; delete/anonymize personal fields when no longer required        | Preserve only records covered by the approved periods or legal/dispute hold. |
| <b>Client identity/contact</b>         | 3 years after last appointment                                         | Delete or irreversibly anonymize personal details                                          | Suspend only when a documented legal/dispute hold applies.                   |
| <b>Private client notes</b>            | 3 years after last appointment                                         | Delete or irreversibly redact/anonymize content                                            | Legal/dispute hold exception; prohibit secondary AI training.                |
| <b>Appointments/booking history</b>    | 3 years after last appointment                                         | Delete or anonymize personal details; retain only separately required evidence             | Financial/audit evidence follows its 7-year rule.                            |
| <b>Payments/refunds/reconciliation</b> | 7 years from financial event                                           | Retain necessary immutable summary/evidence; delete unnecessary personal/raw payload data  | Never store sensitive card/bank data; legal/dispute hold may extend.         |
| <b>Consent evidence</b>                | 3 years after last appointment unless another law/hold requires longer | Delete/anonymize personal details while retaining only necessary evidence                  | Withdrawal adds evidence; does not silently rewrite prior facts.             |
| <b>Message contents</b>                | 3 years after last appointment                                         | Delete or irreversibly redact/anonymize bodies                                             | Delivery fact/digest may remain only when necessary under audit rule.        |
| <b>Audit/security events</b>           | 7 years from event                                                     | Retain standard append-only v1 evidence; delete at end unless legal/dispute hold           | Application cannot update/delete; redact prohibited secrets before storage.  |
| <b>Raw payment webhooks</b>            | 90 days from receipt                                                   | Securely delete raw payload; retain necessary normalized payment/audit summary for 7 years | Encrypt and restrict raw access; quarantine invalid events.                  |
| <b>Idempotency/holds/cache</b>         | Short operational TTL                                                  | Automatic hard deletion after safe retry/settlement window                                 | Not a business-history substitute.                                           |
| <b>Theme assets/versions</b>           | Active + rollback window                                               | Delete unreferenced binaries; retain publication metadata as needed                        | Published historical asset may be archived privately.                        |
| <b>AI transcripts</b>                  | Only when needed for audit/support; 3 years from event                 | Redact before storage; securely delete after 3 years unless legal/dispute hold             | Secondary AI training is prohibited; retain minimal action provenance.       |

## Deletion workflow

- Request -> authenticate/authorize -> classify scope -> check legal/dispute holds -> preview impact -> approve -> execute idempotently -> verify object storage/search/cache deletion -> append audit evidence.
- Account deletion runs as a resumable state machine and blocks new writes. Partial failure is visible and retryable.

# 15. Migration and Rollback Strategy

## Migration principles

- **Expand/contract:** add nullable columns/tables and dual-compatible code first; backfill in bounded account batches; validate; switch reads/writes; remove old structures only in a later release.
- **Forward repair preferred:** schema/data migrations are immutable and checksummed. Destructive down migrations are not the default rollback mechanism.
- **Tenant-safe backfills:** each batch is account-scoped, resumable, idempotent, observable, and checked for orphan/cross-account rows.
- **Online safety:** use concurrent indexes where supported, short lock timeouts, statement timeouts, preflight size estimates, and pause/abort thresholds.
- **Version compatibility:** application versions declare supported schema range. Deployment refuses incompatible combinations.

## Release sequence

| Stage        | Gate                                                                                                           |
|--------------|----------------------------------------------------------------------------------------------------------------|
| 1. Preflight | Backup/PITR health; migration checksum; disk/lock/replica lag; invariant scans; restore point.                 |
| 2. Expand    | Create additive schema, constraints NOT VALID where appropriate, new indexes, RLS policies, feature flags off. |
| 3. Backfill  | Account-scoped batches with progress table, hashes/count reconciliation, throttling and retry.                 |
| 4. Validate  | Validate constraints/RLS, compare old/new read models, isolation tests, payment/scheduling invariants.         |
| 5. Activate  | Canary accounts/internal data, dual read or shadow compare, then controlled flag rollout.                      |
| 6. Contract  | After observation/rollback window, stop old writes, archive mapping, remove obsolete schema in later release.  |

## Rollback levels

- Application rollback while additive schema remains compatible.
- Feature flag rollback to prior action/read path or prior published configuration.
- Forward corrective migration for committed data/schema errors.
- Point-in-time restore only for catastrophic loss/corruption, with recovery environment, reconciliation, and explicit incident decision.

## 16. Backup, Restore, and Recovery Objectives

### Approved future production requirement - not current live infrastructure

These are implementation acceptance requirements for the future production system. This review draft does not assert that deployed backup or recovery infrastructure exists today.

| System                        | Future protection                                                             | Approved objective                                                             | Required proof                                                                |
|-------------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Production database</b>    | Encrypted continuous point-in-time recovery stream plus recoverable snapshots | RPO maximum 1 hour; RTO maximum 4 hours                                        | Automated backup health checks; quarterly full restore; record actual results |
| <b>Object assets</b>          | Versioned encrypted object storage with recoverable metadata                  | Included in maximum 1-hour RPO and 4-hour RTO for required service restoration | Quarterly restore with metadata/digest verification                           |
| <b>Secrets/config</b>         | Managed secret version recovery plus infrastructure as code                   | Included in 4-hour RTO for required service restoration                        | Quarterly recovery exercise; rotate after incident                            |
| <b>Audit/payment evidence</b> | Encrypted backups aligned with approved 7-year retention                      | RPO maximum 1 hour; RTO maximum 4 hours for required evidence access           | Quarterly restore plus record-count/digest reconciliation                     |

### Recovery requirements

- Backups are encrypted, access-controlled, monitored, and logically separated from runtime credentials.
- A restore is not successful until migrations apply, RLS is active, isolation/invariant tests pass, object references reconcile, and payment/provider events are replayed or reconciled safely.
- Quarterly full restore tests record actual RPO/RTO, scope, gaps, owner, remediation deadline, and evidence that the maximum 1-hour RPO and 4-hour RTO were met.
- Automated backup health checks alert on stream interruption, failed snapshots, stale restore points, integrity errors, and insufficient retention. Provider/topology selection remains an implementation choice subject to these targets.

## 17. Validation and Cross-Account Isolation Tests

### Approval gate

Tests must prove that one professional's protected business and client data cannot leak into another account - through reads, writes, relationships, search, events, files, logs, caches, integrations, agents, backups, or administrative tooling.

| ID     | Test                 | Required proof                                                                                                                                                                                                    |
|--------|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ISO-01 | RLS read matrix      | Create accounts A/B with identical-looking records. Under A context, every protected table/query returns only A; direct B IDs return not-found/denied.                                                            |
| ISO-02 | Write/insert scope   | Under A context, insert/update/delete using B account or B parent IDs fails. Verify no partial audit/outbox/payment rows.                                                                                         |
| ISO-03 | Composite FK attack  | Attempt A child -> B client/service/appointment/theme/integration reference; database rejects.                                                                                                                    |
| ISO-04 | Context pooling      | Alternate A/B requests on reused connections; missing/stale transaction context fails closed and never carries scope forward.                                                                                     |
| ISO-05 | Repository fuzz      | Property-based tests feed valid/invalid foreign IDs to every canonical action and list/search endpoint.                                                                                                           |
| ISO-06 | Public projection    | Unpublished/private notes, contacts, policies, themes/assets, and appointments never appear in booking-page reads or asset URLs.                                                                                  |
| ISO-07 | Scheduling isolation | A availability, segments, holds, and appointments never affect B slot calculation, overlap fit, or locks. Public non-fitting overlaps are rejected; professional manual exceptions require A authority and audit. |
| ISO-08 | Payment/webhook      | Provider connection/event/reference for A cannot update B payment or appointment; duplicates and reordered events remain account-bound.                                                                           |
| ISO-09 | Outbox/worker        | Worker claims event globally but sets event account context before load; malformed account/aggregate mismatch quarantines.                                                                                        |
| ISO-10 | Integration scope    | Connection/token for A can invoke only A canonical actions; external references cannot map to B.                                                                                                                  |
| ISO-11 | Agent provenance     | Agent invocation under A cannot query/action B even with a leaked B opaque ID; denied attempt is audited.                                                                                                         |
| ISO-12 | Object storage       | Signed URLs, storage keys, CDN/cache keys, and delete jobs enforce account and publication scope.                                                                                                                 |
| ISO-13 | Search/cache/export  | Search indexes, caches, reports, analytics extracts, CSV exports, and notification jobs remain account-partitioned.                                                                                               |
| ISO-14 | Deletion             | Deleting/anonymizing A cannot alter B and removes A-derived cache/index/object copies; immutable evidence remains correctly scoped.                                                                               |
| ISO-15 | Backup/restore       | Restore to isolated environment; verify RLS/policies/roles, tenant counts/hashes, and run full isolation suite before service.                                                                                    |
| ISO-16 | Privileged access    | Runtime cannot bypass RLS. Migration/break-glass use is logged, alerted, time-limited, and excluded from normal request paths.                                                                                    |

### Exit criteria

- All tests automated in CI against the production database engine and migration set; zero known bypasses.
- Every protected table appears in the RLS/ownership inventory; schema-diff test fails when a new table lacks classification and policy.
- Negative tests cover owner, client/public, integration, agent, worker, and runtime roles.

## 18. Approved Phase Five Decision Log

All listed Phase Five decisions are approved v1 baselines and are binding inputs to Phase Six.

| Decision / status         | Topic                          | Approved baseline or remaining question                                                                                                                                                                                                                                |
|---------------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>P5-D01 - APPROVED</b>  | Database and isolation         | PostgreSQL with composite tenant keys and FORCE RLS is the v1 baseline.                                                                                                                                                                                                |
| <b>P5-D02 - APPROVED</b>  | Identifier strategy            | Sortable UUIDs (UUIDv7/equivalent) are internal keys; friendly human references are separate.                                                                                                                                                                          |
| <b>P5-D03 - APPROVED</b>  | Overlap limits and precedence  | Professional configures maximum overlapping clients during eligible processing time. Public booking blocks limit excess or non-fit. Manual exceptions show a clear warning, require confirmation, and record basis, actor, confirmation, affected segments, and audit. |
| <b>P5-D04 - APPROVED</b>  | No v1 resource tracking        | Remove stations, rooms, waiting areas, equipment, named resources, pools, and capacity semantics from v1. Preserve timed segments.                                                                                                                                     |
| <b>P5-D05 - APPROVED</b>  | Hold and booking timing        | Use a ten-minute hold; release early on client cancel/close, otherwise expire and reopen. Confirm only after required professional-configured payment terms are satisfied.                                                                                             |
| <b>P5-D05A - APPROVED</b> | Payment policy modes           | Professional may require deposit, full prepayment, no prepayment, or provider-held card on file. Sensitive payment data stays with provider.                                                                                                                           |
| <b>P5-D06 - APPROVED</b>  | Retention/anonymization        | Ordinary client/booking history: 3 years after last appointment, then delete/anonymize personal details unless legal/dispute hold. Payment/refund/audit evidence: 7 years.                                                                                             |
| <b>P5-D07 - APPROVED</b>  | Sensitive notes/content        | Client notes and message contents: 3 years after last appointment, then delete/anonymize unless legal/dispute hold.                                                                                                                                                    |
| <b>P5-D08 - APPROVED</b>  | Agent data                     | Store AI transcripts only when needed for audit/support; redact; retain 3 years; prohibit secondary AI training.                                                                                                                                                       |
| <b>P5-D09 - APPROVED</b>  | Audit strength                 | Use standard append-only v1 audit history; retain audit evidence 7 years.                                                                                                                                                                                              |
| <b>P5-D10 - APPROVED</b>  | Recovery objectives            | Future production requirement: RPO maximum 1 hour, RTO maximum 4 hours, continuous PITR, automated health checks, quarterly full restore tests, and measured proof.                                                                                                    |
| <b>P5-D11 - APPROVED</b>  | Support access                 | No default access. Break-glass only with explicit professional consent, recorded reason, short time limit, and full audit.                                                                                                                                             |
| <b>P5-D12 - APPROVED</b>  | Payment providers/raw webhooks | Retain encrypted raw payment webhooks 90 days; then keep only necessary normalized payment/audit summary for 7 years. Provider selection remains later.                                                                                                                |

### Decision status

All Phase Five decisions in this log are approved. Phase Five has no deferred database-policy item remaining.

## 19. Blueprint Deliverables Traceability

| Required deliverable                        | Location      | Status                                                                                                         |
|---------------------------------------------|---------------|----------------------------------------------------------------------------------------------------------------|
| Conceptual and physical ERDs                | Sections 3-10 | Conceptual entities plus implementable table/key outline; v1 is time-based with no resource/capacity entities. |
| Table/column specification                  | Sections 5-10 | Representative columns, keys, ownership, lifecycle, and invariants.                                            |
| Keys, constraints, indexes, exclusion rules | Section 12    | Composite tenant integrity, temporal/money/version/idempotency/provider/scheduling rules.                      |
| Account isolation and RLS policies          | Section 11    | Seven-layer authorization/isolation model and role posture.                                                    |
| Migration and rollback                      | Section 15    | Expand/contract, backfill, validation, activation, forward repair, catastrophic restore.                       |
| Retention/archive/deletion matrix           | Section 14    | Approved 3-year, 7-year, and 90-day rules plus legal/dispute hold handling.                                    |
| Backup and restore objectives               | Section 16    | Approved future production maximum 1-hour RPO and 4-hour RTO with required proof.                              |
| Isolation proof                             | Section 17    | Sixteen explicit cross-account validation tests and exit criteria.                                             |

### Phase Six handoff contract

- API/action design must use account-scoped identifiers, canonical action transactions, idempotency, stable errors, and actor/agent/integration provenance.
- No action may bypass Scheduling for slot/segment writes, Payment for financial state, Theme for publication, or Audit/Outbox for consequential evidence/events.
- Phase Six contracts must encode the approved 3-year/7-year/90-day retention rules, redacted audit-only AI transcripts, no-default support access, consented break-glass, and future recovery evidence requirements.
- Physical DDL and migrations must persist the configured maximum overlap, public block outcome, and manual warning basis, actor, explicit confirmation, affected segments, and audit evidence.

## 20. Phase 5 Approval Gate

**STATUS: PHASE FIVE APPROVED - ALL LISTED DECISIONS RESOLVED**

### Approved validation checklist

- Does the business-account tenant boundary correctly support one independent professional and many clients without introducing team/staff workflows?
- Can the composite-key, authorization, and RLS design prove that protected data never crosses accounts?
- Does the service-segment time model support intentional double-booking without prescribing or requiring operational resource tracking?
- Does public booking block any overlap exceeding the professional-configured maximum or whose full service does not fit the eligible processing window?
- Do appointments preserve the service, price, timing, policy, overlap, and consent facts that were true when booked or changed?
- Do professional-configured deposit, full-prepayment, no-prepayment, and provider-held card-on-file modes confirm only after required terms are satisfied, with reliable retries, refunds, reconciliation, and no sensitive payment data stored?
- Are theme tokens/assets/versioning expressive enough while structurally preventing changes to core flow, labels, layout, prices, timing, policies, and consent facts?
- Are integrations and agents provably limited to canonical actions with complete provenance and no direct database access?
- Do retention actions enforce 3-year client/booking/content rules, 7-year payment/refund/audit evidence, 3-year redacted audit/support AI transcripts with no training, and 90-day raw payment webhooks?
- Do future production controls prove maximum 1-hour RPO and 4-hour RTO through continuous PITR, automated health checks, and quarterly full restore evidence?
- Does support remain closed by default, with consented, reasoned, short-lived, fully audited break-glass access only?
- Does every professional manual exception show a clear warning, require explicit confirmation before save, and record warning basis, actor, confirmation, affected segments, and audit evidence?

#### Approval effect

Phase Five is fully approved as binding input to Phase Six API & Action Design, including the professional-configured overlap maximum and manual-exception evidence contract. This approval does not authorize production implementation, final vendor selection, production data processing, Phase Seven UI approval, or launch.

#### Final review gate

Can tests prove that one professional's protected business and client data cannot leak into another account, while every consequential record remains explainable, recoverable, and historically truthful?